

实 验 报 告

课程名称：模型评估方法的对比与
Transformer 的从零实现

姓名：王志豪

学号：08232337

班级：2023 大数据一班

日期：2026. 5. 12

中国矿业大学计算机学院/人工智能学院

实验名称 模型评估方法对比与 Transformer 的从零实现 实验日期 2026. 5. 12

实验报告内容: 1. 实验目的 2. 实验内容 3. 实验步骤 4. 实验结果 5. 实验体会

一、实验目的

通过本次实验我将设计两个机器学习实验，分别是：**模型评估方法的综合对比**、**从零实现一个 Transformer 编码器分类模型**

对于**模型评估方法的综合对比**：

- 掌握留出法与 K 折交叉验证以及自助法的评估流程与结果解读方法
- 综合对比三种模型评估数据集划分方法的效果
- 学会利用学习曲线分析模型的偏差/方差问题
- 能基于评估结果提出模型改进方向（特征工程、正则化等）

对于**Transformer 编码器分类模型**：

- 理解二分类文本分类任务的基本流程，包括文本清洗、分词、词表构建、填充截断 attention_mask 生成。
- 从零实现 Transformer 编码器分类模型，掌握自注意力、多头注意力、位置编码、前馈网络、残差连接与层归一化等核心结构。
- 使用 Hugging Face Transformers 对 bert-base-uncased 进行微调，并与从零训练的 Transformer 进行对比。
- 构建传统机器学习基线模型 TF-IDF + LogisticRegression，理解传统特征方法与深度模型的差异。
- 使用 Accuracy、F1、混淆矩阵和训练曲线分析模型性能、泛化能力与过拟合现象。

二、实验内容

2.1 基于 California Housing Prices 数据集，训练线性回归模型，并分别测试留出法、交叉验证和自助法的数据集划分效果，并对两种方法的结果进行归纳得出相应。

A. 数据集准备与预处理

- 使用 California Housing 数据集
- 若存在 total_bedrooms 缺失值，删除对应行
- 若存在 ocean_proximity，进行手动独热编码
- 数值特征标准化： $x' = \frac{(x - \mu)}{\sigma}$
- 按 70%/30% 划分训练集与测试集（固定随机种子）

B. 留出法评估

- 线性回归采用正规方程闭式解
- 在测试集上计算 R^2 与 MSE
- 结果显示模型有一定拟合能力，但误差仍偏高

C. K 折交叉验证

- 使用固定随机种子进行 $K=5$ 与 $K=10$ 划分
- 每折训练线性回归并评估验证集
- 记录各折指标并计算均值与标准差
- 交叉验证结果与留出法接近，但稳定性更强

D. 学习曲线绘制

- 使用 10% ~ 100% 的训练子集
- 对每个子集进行 5 折 CV，得到训练/验证 MSE
- 曲线显示两条误差在 50% 之后逐渐收敛且差距较小
- 说明模型偏差较高（欠拟合）

E. 自助法评估

- 有放回采样构造训练集，未被采样的样本作为 OOB 验证集
- 重复 B 次，统计 OOB 上的 R^2 与 MSE 均值/标准差
- Bootstrap 的 R^2 基于每次采样的 OOB 验证集

F. 正则化影响

- 在 $10^{-4} \sim 10^3$ 的 λ 取值范围内评估
- 小 λ 时验证误差变化不大，大 λ 时误差明显升高
- 表明正则化对该线性模型提升有限

2.2 从零实现一个 Transformer 编码器分类模型

A. IMDB 数据集的加载与划分

- 使用 IMDB 数据集
- 从原始训练集中划分验证集；实际使用训练集 12000 条、验证集 2000 条、测试集 5000 条

B. 完成文本的预处理

- HTML 标签去除、小写化、特殊字符过滤、正则分词、词表构建、序列截断填充、attention_mask 构造

C. 从零实现 Transformer 编码器文本分类模型

- 不使用 nn.TransformerEncoderLayer
- 手动实现：多头自注意力、位置编码、前馈网络、残差连接、层归一化、分类头
- 训练时记录 loss 与 accuracy，使用验证集监控，早停保存最优模型

D. 微调 BERT

- 使用 bert-base-uncased 预训练模型和分词器
- 在同一数据划分上完成情感分类微调

E. 构建传统机器学习基线模型

- 使用 TfidfVectorizer(max_features=10000) 提取特征
- 使用逻辑回归完成分类

F. 对三种模型进行统一评估

- 评估指标：Accuracy、F1、混淆矩阵

- 绘制训练曲线和混淆矩阵图像

三、实验原理

(1). 模型评估方法的综合对比

A. 留出法

留出法是最简单、最常用的模型评估方法。它将原始数据集 D 划分为两个互斥的集合：**训练集** D_{train} 和 **测试集** D_{test} 。通常训练集占 60% ~ 80%，剩余作为测试集。

模型在训练集上学习参数，在测试集上评估泛化性能。测试集不参与训练，因此测试误差可作为对未知数据泛化误差的无偏估计（假设数据独立同分布）。

设数据集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}$ ，共有 N 个样本。按比例 p （如 $p = 0.7$ ）随机划分：

$$|D_{\text{train}}| = \lfloor p \cdot N \rfloor, |D_{\text{test}}| = N - |D_{\text{train}}|$$

模型 f 在 D_{train} 上训练得到参数 θ ，然后在 D_{test} 上计算评估指标（如准确率、 R^2 等）：

$$E_{\text{test}} = \frac{1}{|D_{\text{test}}|} \sum_{(\mathbf{x}_i, y_i) \in D_{\text{test}}} \ell(f(\mathbf{x}_i; \theta), y_i)$$

其中 ℓ 为损失函数。

B. K 折交叉验证

K 折交叉验证将数据集 D 随机划分为 K 个大小近似相等的子集（折），记为 $D_1, D_2, \dots, D_K$ 。进行 K 次训练与评估：

- 在第 i 次迭代中，将 D_i 作为验证集（测试集），其余 $K - 1$ 个子集的并集作为训练集。
- 训练模型后，在 D_i 上计算评估指标 E_i 。
- 最终性能为 K 次结果的平均值：

$$E_{\text{cv}} = \frac{1}{K} \sum_{i=1}^K E_i$$

通常也计算标准差来衡量评估的稳定性：

$$\sigma = \sqrt{\frac{1}{K-1} \sum_{i=1}^K (E_i - E_{\text{cv}})^2}$$

设 D_i 为第 i 折， $D_{-i} = D \setminus D_i$ 为其余样本。

训练模型 $f_{\theta^{(i)}}$ 使用 D_{-i} ，误差为：

$$E_i = \frac{1}{|D_i|} \sum_{(\mathbf{x}_j, y_j) \in D_i} \ell(f(\mathbf{x}_j; \theta^{(i)}), y_j)$$

最终估计：

$$\widehat{\text{Err}} = \frac{1}{K} \sum_{i=1}^K E_i$$

C. 自助法

设原始数据集 D 包含 N 个样本。自助法进行 B 次（通常 $B = 100 \sim 1000$ ）如下操作：

- 从 D 中有放回地随机抽取 N 个样本，形成自助样本 $D^{(b)}$ （训练集）。
- 将未出现在 $D^{(b)}$ 中的样本（称为“袋外数据”，out-of-bag, OOB）作为验证集 $D_{\text{OOB}}^{(b)}$ 。
- 在 $D^{(b)}$ 上训练模型，在 $D_{\text{OOB}}^{(b)}$ 上评估性能。
- 最终性能为 B 次评估结果的平均值，并可计算标准差及置信区间。

由于是有放回抽样，一个样本在单次自助抽样中未被选中的概率为：

$$\left(1 - \frac{1}{N}\right)^N \xrightarrow{N \rightarrow \infty} \frac{1}{e} \approx 0.36$$

因此，每次自助采样得到的训练集平均包含约 63.2% 的原始样本，其余约 36.8% 的样本作为袋外验证集。这使得自助法与留出法（通常 70%/30% 划分）在训练/验证比例上具有一定的可比性。

设原始数据集 $D = \{\mathbf{z}_1, \mathbf{z}_2, \dots, \mathbf{z}_N\}$ ，其中 $\mathbf{z}_i = (\mathbf{x}_i, y_i)$ 。对于第 b 次自助采样：

- 生成自助样本 $D^{(b)} = \{\mathbf{z}_{j_1}^{(b)}, \mathbf{z}_{j_2}^{(b)}, \dots, \mathbf{z}_{j_N}^{(b)}\}$ ，其中每个 $j_k^{(b)}$ 从 $\{1, 2, \dots, N\}$ 中均匀随机抽取（有放回）。
- 袋外数据集 $D_{\text{OOB}}^{(b)} = D \setminus D^{(b)}$ （即未被抽中的原始样本）。

训练模型得到参数 $\theta^{(b)}$ ，则第 b 次评估误差为：

$$E^{(b)} = \frac{1}{|D_{\text{OOB}}^{(b)}|} \sum_{\mathbf{z}_i \in D_{\text{OOB}}^{(b)}} \ell(f(\mathbf{x}_i; \theta^{(b)}), y_i)$$

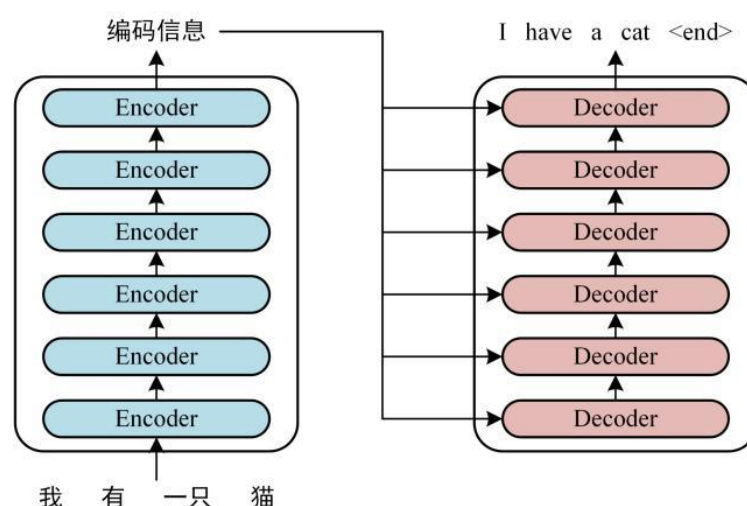
自助法最终评估指标：

$$E_{\text{bootstrap}} = \frac{1}{B} \sum_{b=1}^B E^{(b)}$$

误差的标准差估计为：

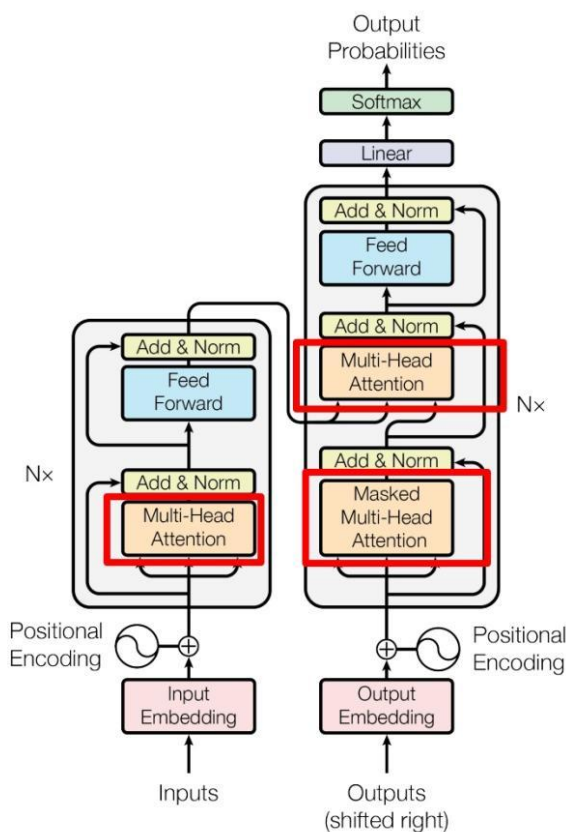
$$\hat{\sigma} = \sqrt{\frac{1}{B-1} \sum_{b=1}^B (E^{(b)} - E_{\text{bootstrap}})^2}$$

(2). 从零实现 Transformer 编码器文本分类模型（这里重点介绍 Transformer 的模型架构）



这里首先介绍一下原始的 Transformer 模型的架构，原始的模型主要由两个核心组件构成：Encoder 和 Decoder，笼统来说，在翻译的任务中，Encoder 是用来理解原句意思的组件，Decoder 是用来综合原句意

思来实现结果句生成的组件。上述的这一张图可以看到，一个编码器中有 6 个 Encoder 组件，一个解码器中有 6 个 Decoder 组件，它们共同组成了 Transformer。



这里的第二张图体现的就是具体的模型架构，这里详细论述几个核心的组件：位置编码、Self-Attention、Add&Norm、Feed Forward：

1) 位置编码

Transformer 的强大之处就在于它不像 RNN 那样只能关注时序的顺序信息，它使用的是全局信息，所以 Transformer 中使用位置 Embedding 保存单词在序列中的相对或绝对位置，位置 Embedding 用 PE 表示，PE 的维度与单词 Embedding 是一样的。PE 可以通过训练得到，也可以使用某种公式计算得到。在 Transformer 中采用了后者，计算公式如下：

$$PE_{(pos, 2i)} = \sin(pos/10000^{2i/d})$$

$$PE_{(pos, 2i+1)} = \cos(pos/10000^{2i/d})$$

其中，pos 表示单词在句子中的位置，d 表示 PE 的维度 (与词 Embedding 一样)，2i 表示偶数的维度，2i+1 表示奇数维度 (即 $2i \leq d, 2i+1 \leq d$)。使用这种公式计算 PE 有以下的好处：

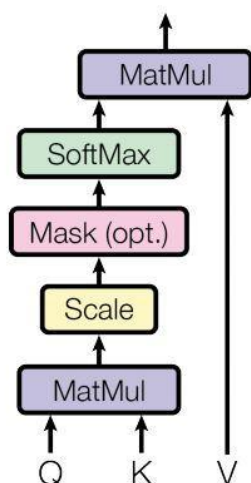
- 使 PE 能够适应比训练集里面所有句子更长的句子，假设训练集里面最长的句子是有 20 个单词，突然来了一个长度为 21 的句子，则使用公式计算的方法可以计算出第 21 位的 Embedding。
- 可以让模型容易地计算出相对位置，对于固定长度的间距 k， $PE(pos+k)$ 可以用 $PE(pos)$ 计算得到。

因为 $\sin(A+B) = \sin(A)\cos(B) + \cos(A)\sin(B)$, $\cos(A+B) = \cos(A)\cos(B) - \sin(A)\sin(B)$ 。

将单词的词 Embedding 和位置 Embedding 相加，就可以得到单词的表示向量 x ， x 就是 Transformer 的输入。

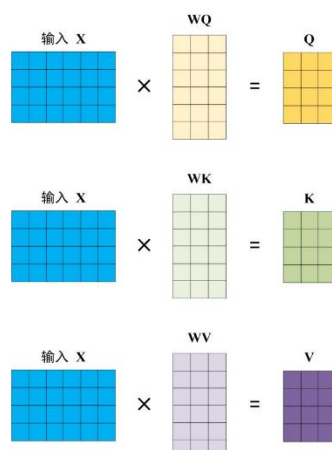
2) Self-Attention

Scaled Dot-Product Attention



上图是 Self-Attention 的结构，在计算的时候需要用到矩阵 **Q(查询)**, **K(键值)**, **V(值)**。在实际中，Self-Attention 接收的是输入(单词的表示向量 x 组成的矩阵 X) 或者上一个 Encoder block 的输出。而 **Q, K, V** 正是通过 Self-Attention 的输入进行线性变换得到的。

Self-Attention 的输入用矩阵 X 进行表示，则可以使用线性变阵矩阵 **WQ, WK, WV** 计算得到 **Q, K, V**：



得到矩阵 Q, K, V 之后就可以计算出 Self-Attention 的输出，计算的公式如下：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

d_k 是 Q, K 矩阵的列数，即向量维度

公式中计算矩阵 Q 和 K 每一行向量的内积，为了防止内积过大，因此除以 $\sqrt{d_k}$ 的平方根。 Q 乘以 K 的转置后，得到的矩阵行列数都为 n ， n 为句子单词数，这个矩阵可以表示单词之间的 attention 强度。

3) Add & Norm

Add & Norm 层由 Add 和 Norm 两部分组成，其计算公式如下：

$$\begin{aligned} & \text{LayerNorm}(X + \text{MultiHeadAttention}(X)) \\ & \text{LayerNorm}(X + \text{FeedForward}(X)) \end{aligned}$$

其中 X 表示 Multi-Head Attention 或者 Feed Forward 的输入，MultiHeadAttention(X) 和 FeedForward(X) 表示输出 (输出与输入 X 维度是一样的，所以可以相加)。

Add 是一种残差链接的方式，用于解决多层网络训练的问题，可以让网络只关注当前差异的部分，在 ResNet 中经常用到。

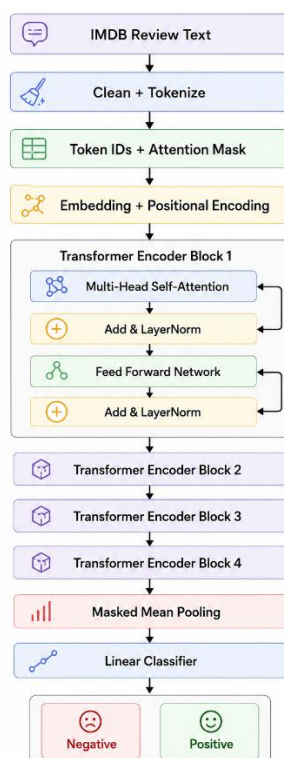
Norm 指 Layer Normalization，通常用于 RNN 结构，Layer Normalization 会将每一层神经元的输入都转成均值方差都一样的，这样可以加快收敛。

4) Feed Forward

Feed Forward 是一个两层的全连接层，第一层的激活函数为 Relu，第二层不使用激活函数，对应的公式如下。 $\mathbf{X}$ 是输入，Feed Forward 最终得到的输出矩阵的维度与 $\mathbf{X}$ 一致。

$$\max(0, \mathbf{X}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2$$

介绍完官方的模型架构，这里来介绍一下本次实验的模型架构，由于只是完成文本分类这种任务，因此 Decoder 其实是完全不需要的，以下是模型的架构图：



可以看到，相比于官方的原始模型，该模型就是简单地实现了 Encoder 组件，并且简化了 Encoder 组件的内部结构，只选取了 4 个 Block，其余的结构原理可以完全参考前文对于官方模型结构的论述。

四、实验步骤

1. 模型评估方法的综合对比：

(1). 数据集准备与预处理

本实验首先加载 California Housing 数据集。程序优先使用 sklearn.datasets.fetch_california_housing 获取数据。该过程由 load_dataset() 完成


```
def load_dataset():
    """Load California Housing dataset.

    Tries sklearn fetch first (data only). Falls back to local housing.csv if available.
    """
    try:
        from sklearn.datasets import fetch_california_housing

        data = fetch_california_housing(as_frame=True)
        df = data.frame.copy()
        return df
    except Exception:
        # Fallback: user-provided CSV, expected to be housing.csv in current directory
        df = pd.read_csv("housing.csv")
        return df
```

随后，程序对数据进行基础清洗与特征处理：若数据集中存在 `total_bedrooms` 字段，则删除该字段缺失的样本；若存在 `ocean_proximity` 类别特征，则调用手动独热编码函数进行编码。

```
# Drop rows with missing total_bedrooms if present
if "total_bedrooms" in df.columns:
    df = df.dropna(subset=["total_bedrooms"])

# If ocean_proximity exists, apply manual one-hot encoding
if "ocean_proximity" in df.columns:
    df = one_hot_encode(df, "ocean_proximity")

def one_hot_encode(df, column):
    """Manual one-hot encoding for a single categorical column."""
    categories = sorted(df[column].dropna().unique())
    for cat in categories:
        df[f"{column}__{cat}"] = (df[column] == cat).astype(float)
    df = df.drop(columns=[column])
    return df
```

接着，程序自动识别目标变量：若数据集中存在 `MedHouseVal`，则将其作为目标列；否则使用 `median_house_value`。完成目标列识别后，将其余字段转换为特征矩阵 `X`，目标列转换为标签向量 `y`。

```
# Separate features and target
if "MedHouseVal" in df.columns:
    target_col = "MedHouseVal"
elif "median_house_value" in df.columns:
    target_col = "median_house_value"
else:
    raise ValueError("Target column not found.")

X = df.drop(columns=[target_col]).to_numpy(dtype=float)
y = df[target_col].to_numpy(dtype=float)
```

最后，对所有特征进行标准化处理，使其均值为 0、标准差为 1。

```
# Standardize features
X, X_mean, X_std = standardize_features(X)
```

```
def standardize_features(X):
    """Standardize features to mean 0 and std 1 (manual)."""
    mean = X.mean(axis=0)
    std = X.std(axis=0, ddof=0)
    # Avoid division by zero
    std_safe = np.where(std == 0, 1.0, std)
    X_std = (X - mean) / std_safe
    return X_std, mean, std_safe
```

(2). 留出法评估

在完成预处理后，实验采用留出法将数据集划分为训练集和测试集，其中测试集比例为 30%，训练集比例为 70%，并固定随机种子 seed=42 以保证结果可复现。

```
def train_test_split(X, y, test_size=0.3, seed=42, shuffle=True):
    n = X.shape[0]
    rng = np.random.default_rng(seed)
    indices = np.arange(n)
    if shuffle:
        rng.shuffle(indices)
    split = int(n * (1.0 - test_size))
    train_idx = indices[:split]
    test_idx = indices[split:]
    return X[train_idx], X[test_idx], y[train_idx], y[test_idx]
```

```
# 2) Hold-out evaluation
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, seed=42, shuffle=True)
```

随后，程序使用正规方程闭式解训练线性回归模型。

```
w = fit_linear_regression_normal_eq(X_train, y_train)
```

```
def fit_linear_regression_normal_eq(X, y, l2_lambda=0.0):
    """Closed-form solution for linear regression (optional L2 regularization)."""
    Xb = add_bias(X)
    n_features = Xb.shape[1]
    I = np.eye(n_features)
    I[0, 0] = 0.0 # Do not regularize bias
    A = Xb.T @ Xb + l2_lambda * I
    b = Xb.T @ y
    w = np.linalg.pinv(A) @ b
    return w
```

留出法评估阶段在测试集上计算预测结果，并使用 R^2 与 MSE 作为评价指标。

```
y_test_pred = predict(X_test, w)
holdout_r2 = r2_score(y_test, y_test_pred)
holdout_mse = mse(y_test, y_test_pred)
```

```
def predict(X, w):
    Xb = add_bias(X)
    return Xb @ w
```

其中 R^2 与 MSE 的计算定义函数如下：

```
def mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

def r2_score(y_true, y_pred):
    ss_res = np.sum((y_true - y_pred) ** 2)
    ss_tot = np.sum((y_true - np.mean(y_true)) ** 2)
    return 1.0 - ss_res / ss_tot
```

(3). K 折交叉验证

实验手动实现 K 折交叉验证，没有使用 `sklearn.model_selection` 等高层 API。程序首先通过 `k_fold_indices()` 生成 K 折划分：对样本索引进行随机打乱，再使用 `np.array_split` 切分为 K 份，每次取其中一份作为验证集，其余部分作为训练集。

```
def k_fold_indices(n, k=5, seed=42, shuffle=True):
    """Return list of (train_idx, val_idx) for K-fold CV."""
    indices = np.arange(n)
    if shuffle:
        rng = np.random.default_rng(seed)
        rng.shuffle(indices)
    folds = np.array_split(indices, k)
    splits = []
    for i in range(k):
        val_idx = folds[i]
        train_idx = np.hstack([folds[j] for j in range(k) if j != i])
        splits.append((train_idx, val_idx))
    return splits
```

交叉验证的完整流程由 `cross_validate()` 实现，每一折中，程序根据索引取出训练集和验证集，训练线性回归模型，在验证集上预测，并分别记录该折的 R^2 与 MSE 。

```
def cross_validate(X, y, k=5, seed=42, shuffle=True, l2_lambda=0.0):
    """Manual K-fold cross validation for linear regression."""
    r2_list = []
    mse_list = []
    for train_idx, val_idx in k_fold_indices(len(y), k=k, seed=seed, shuffle=shuffle):
        X_train, y_train = X[train_idx], y[train_idx]
        X_val, y_val = X[val_idx], y[val_idx]
        w = fit_linear_regression_normal_eq(X_train, y_train, l2_lambda=l2_lambda)
        y_pred = predict(X_val, w)
        r2_list.append(r2_score(y_val, y_pred))
        mse_list.append(mse(y_val, y_pred))
    return np.array(r2_list), np.array(mse_list)
```

主函数中分别设置 $K=5$ 和 $K=10$ 进行交叉验证。程序输出每种 K 值下 R^2 与 MSE 的均值与标准差，用于比较不同划分方式下模型评估结果的稳定性。

```
# 3) K-fold cross validation
for k in [5, 10]:
    r2_list, mse_list = cross_validate(X, y, k=k, seed=42, shuffle=True)
    print(f"\n{k}-fold CV results:")
    print(f"  R2 mean = {r2_list.mean():.4f}, std = {r2_list.std():.4f}")
    print(f"  MSE mean = {mse_list.mean():.4f}, std = {mse_list.std():.4f}")
```

(4). Bootstrap 自助法评估

每次 Bootstrap 采样中，程序从原始样本中进行有放回抽样，得到训练样本；未被抽中的样本作为 OOB 验证集。Bootstrap 的函数定义如下：

```
def bootstrap_evaluate(X, y, n_bootstrap=100, seed=42):
    """Bootstrap evaluation using out-of-bag (OOB) samples."""
    n = len(y)
    rng = np.random.default_rng(seed)
    r2_list = []
    mse_list = []

    for _ in range(n_bootstrap):
        sample_idx = rng.integers(0, n, size=n)
        oob_mask = np.ones(n, dtype=bool)
        oob_mask[sample_idx] = False
        oob_idx = np.where(oob_mask)[0]

        if oob_idx.size == 0:
            continue

        X_train, y_train = X[sample_idx], y[sample_idx]
        X_oob, y_oob = X[oob_idx], y[oob_idx]

        w = fit_linear_regression_normal_eq(X_train, y_train)
        y_pred = predict(X_oob, w)
        r2_list.append(r2_score(y_oob, y_pred))
        mse_list.append(mse(y_oob, y_pred))

    return np.array(r2_list), np.array(mse_list)
```

在每次重复中，程序使用 Bootstrap 训练集训练线性回归模型，并在 OOB 样本上计算 R^2 与 MSE。主函数中设置 `n_bootstrap=100`，调用与输出过程如下。

```
# 4) Bootstrap evaluation (OOB)
boot_r2, boot_mse = bootstrap_evaluate(X, y, n_bootstrap=100, seed=42)
print("\nBootstrap (OOB) results:")
print(f" R2 mean = {boot_r2.mean():.4f}, std = {boot_r2.std():.4f}")
print(f" MSE mean = {boot_mse.mean():.4f}, std = {boot_mse.std():.4f}")
```

(5). 学习曲线绘制

实验通过学习曲线分析模型的偏差与方差情况。学习曲线函数 `learning_curve()` 定义如下。

```
def learning_curve(X, y, train_sizes, k=5, seed=42, shuffle=True):
    """Compute training and validation errors for different train sizes."""
    n = X.shape[0]
    rng = np.random.default_rng(seed)
    indices = np.arange(n)
    if shuffle:
        rng.shuffle(indices)

    train_mse_means = []
    val_mse_means = []

    for frac in train_sizes:
        size = max(2, int(n * frac))
        subset_idx = indices[:size]
        X_sub = X[subset_idx]
        y_sub = y[subset_idx]

        # For each size, run K-fold CV on the subset
        train_mse_folds = []
        val_mse_folds = []
        for train_idx, val_idx in k_fold_indices(len(y_sub), k=k, seed=seed, shuffle=shuffle):
            X_train, y_train = X_sub[train_idx], y_sub[train_idx]
            X_val, y_val = X_sub[val_idx], y_sub[val_idx]
            w = fit_linear_regression_normal_eq(X_train, y_train)
            y_train_pred = predict(X_train, w)
            y_val_pred = predict(X_val, w)
            train_mse_folds.append(mse(y_train, y_train_pred))
            val_mse_folds.append(mse(y_val, y_val_pred))

        train_mse_means.append(np.mean(train_mse_folds))
        val_mse_means.append(np.mean(val_mse_folds))

    return np.array(train_mse_means), np.array(val_mse_means)
```

程序首先设置不同训练集比例，然后在每个训练子集上进行 5 折交叉验证，分别计算训练 MSE 与验证 MSE 。训练子集比例的生成如下，其中 `np.linspace(0.1, 1.0, 10)` 表示从 10% 到 100% 选取 10 个训练规模。

```
# 5) Learning curve
train_sizes = np.linspace(0.1, 1.0, 10)
train_mse, val_mse = learning_curve(X, y, train_sizes, k=5, seed=42, shuffle=True)
```

学习曲线绘制使用 Matplotlib 完成，程序绘制训练 MSE 与验证 MSE 随训练集规模变化的曲线，具体实现如下：

```
plt.figure(figsize=(8, 5))
plt.plot(train_sizes * 100, train_mse, marker="o", label="Train MSE")
plt.plot(train_sizes * 100, val_mse, marker="s", label="Validation MSE")
plt.xlabel("Training set size (%)")
plt.ylabel("MSE")
plt.title("Learning Curve (Linear Regression)")
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()
```

(6). 正则化影响分析

实验进一步分析 L_2 正则化对线性回归模型的影响。正规方程求解函

数 `fit_linear_regression_normal_eq()` 中包含参数 `L2_λ`，当该参数大于 0 时，相当于进行岭回归。实现如下，其中构造了正则化矩阵并加入 λ ，且明确不对偏置项进行正则化。

```
def fit_linear_regression_normal_eq(X, y, l2_lambda=0.0):
    """Closed-form solution for linear regression (optional L2 regularization)."""
    Xb = add_bias(X)
    n_features = Xb.shape[1]
    I = np.eye(n_features)
    I[0, 0] = 0.0 # Do not regularize bias
    A = Xb.T @ Xb + l2_lambda * I
    b = Xb.T @ y
    w = np.linalg.pinv(A) @ b
    return w
```

主函数中使用 `np.logspace(-4, 3, 10)` 在 10^{-4} 到 10^3 范围内生成不同的 λ 值。随后对每个 λ 进行 5 折交叉验证，并记录验证 MSE 均值。

```
# 6) Optional: Ridge regression impact
lambdas = np.logspace(-4, 3, 10)
ridge_val_mse = []
for lam in lambdas:
    _, mse_list = cross_validate(X, y, k=5, seed=42, shuffle=True, l2_lambda=lam)
    ridge_val_mse.append(mse_list.mean())
```

最后，程序绘制 λ 与验证 MSE 的关系曲线，用于观察正则化强度变化对模型性能的影响。

```
plt.figure(figsize=(8, 5))
plt.semilogx(lambdas, ridge_val_mse, marker="o")
plt.xlabel("Lambda (L2)")
plt.ylabel("Validation MSE")
plt.title("Ridge Regression: Lambda vs Validation MSE")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

2. 模型评估方法的综合对比：

(1). 初始化实验配置

程序创建 `Config` 配置对象，设置随机种子、输出目录、数据规模、模型超参数、训练轮数、学习率和 `batch size` 等参数。

类别	参数名	值	说明
基础配置	seed	42	随机种子
	save_dir	"outputs"	输出保存目录
数据与预处理	max_vocab_size	20000	最大词汇表大小
	min_freq	2	词频最低阈值
	max_length	256	序列最大长度（截断/填充）

	val_ratio	0.1	验证集划分比例
	train_limit	12000	训练集样本限制（子集，None 表示全量）
	val_limit	2000	验证集样本限制
	test_limit	5000	测试集样本限制
Transformer	batch_size	32	批大小
	transformer_epochs	10	训练轮数
	transformer_lr	1e-3	学习率
	transformer_weight_decay	1e-4	权重衰减系数
	d_model	128	模型隐藏层维度
	n_heads	4	多头注意力头数
	n_layers	4	Encoder 层数
	ff_dim	256	前馈网络中间层维度
	dropout	0.1	Dropout 比率
	early_stop_patience	3	早停耐心值
BERT 微调	bert_model_name	"bert-base-uncased"	BERT 预训练模型名称
	bert_batch_size	16	批大小
	bert_epochs	3	训练轮数
	bert_lr	2e-5	学习率
DataLoader	num_workers	0	数据加载子进程数（0 表示主进程加载）


```

class Config:
    """实验配置类，集中管理超参数与路径。"""
    seed: int = 42
    save_dir: str = "outputs"

    # 数据与预处理
    max_vocab_size: int = 20000
    min_freq: int = 2
    max_length: int = 256
    val_ratio: float = 0.1

    # 为了保证 CPU 也能跑通，默认使用子集；如需全量训练可设为 None
    train_limit: Optional[int] = 12000
    val_limit: Optional[int] = 2000
    test_limit: Optional[int] = 5000

    # 从零实现 Transformer
    batch_size: int = 32
    transformer_epochs: int = 10
    transformer_lr: float = 1e-3
    transformer_weight_decay: float = 1e-4
    d_model: int = 128
    n_heads: int = 4
    n_layers: int = 4
    ff_dim: int = 256
    dropout: float = 0.1
    early_stop_patience: int = 3

    # BERT 微调
    bert_model_name: str = "bert-base-uncased"
    bert_batch_size: int = 16
    bert_epochs: int = 3
    bert_lr: float = 2e-5

    # DataLoader
    num_workers: int = 0

```

(2). 创建输出目录

程序自动创建 `outputs`、`outputs/models` 和 `outputs/figures`，分别用于保存词表、模型权重、训练曲线和混淆矩阵图片。

```

os.makedirs(config.save_dir, exist_ok=True)
model_dir = os.path.join(config.save_dir, "models")
figure_dir = os.path.join(config.save_dir, "figures")
os.makedirs(model_dir, exist_ok=True)
os.makedirs(figure_dir, exist_ok=True)

```

(3). 检测运行设备

程序通过 `torch.cuda.is_available()` 判断是否存在可用 GPU，本次实验实际使用设备为 `cuda`。

```

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"当前设备: {device}")

```

(4). 加载数据集

程序优先调用 `load_dataset("imdb")` 加载 IMDB；若下载失败，脚本中还提供 SST-2 和 AG News 的备用方案。本次实际使用的是 IMDB。

```
dataset_name, train_texts_raw, train_labels, val_texts_raw, val_labels, test_texts_raw, test_labels = load_binary_text_dataset(config)
print(f"实际使用数据集: {dataset_name}")
print(f"训练集大小: {len(train_texts_raw)}, 验证集大小: {len(val_texts_raw)}, 测试集大小: {len(test_texts_raw)}")
```

封装的数据集加载函数如下所示，使用 `train_test_split` 划分训练集和验证集：

```
def load_binary_text_dataset(config: Config) -> Tuple[str, List[str], List[int], List[str], List[int], List[str], List[int]]:
    """
    加载二分类文本数据。
    返回:
        dataset_name, train_texts, train_labels, val_texts, val_labels, test_texts, test_labels
    """
    dataset_name = ""
    raw_train_texts, raw_train_labels = [], []
    raw_test_texts, raw_test_labels = [], []

    try:
        imdb = load_dataset("imdb")
        dataset_name = "IMDB"
        raw_train_texts = [item["text"] for item in imdb["train"]]
        raw_train_labels = [int(item["label"]) for item in imdb["train"]]
        raw_test_texts = [item["text"] for item in imdb["test"]]
        raw_test_labels = [int(item["label"]) for item in imdb["test"]]
    except Exception as e1:
        print(f"IMDB 下载失败, 开始尝试 SST-2。错误信息: {e1}")
        try:
            sst2 = load_dataset("glue", "sst2")
            dataset_name = "SST-2"
            raw_train_texts = [item["sentence"] for item in sst2["train"]]
            raw_train_labels = [int(item["label"]) for item in sst2["train"]]
            raw_test_texts = [item["sentence"] for item in sst2["validation"]]
            raw_test_labels = [int(item["label"]) for item in sst2["validation"]]
        except Exception as e2:
            print(f"SST-2 下载失败, 开始尝试 AG News (二值化)。错误信息: {e2}")
            ag_news = load_dataset("ag_news")
            dataset_name = "AG News (binary)"
            raw_train_texts = [item["text"] for item in ag_news["train"]]
            raw_train_labels = [0 if int(item["label"]) < 2 else 1 for item in ag_news["train"]]
            raw_test_texts = [item["text"] for item in ag_news["test"]]
            raw_test_labels = [0 if int(item["label"]) < 2 else 1 for item in ag_news["test"]]

    train_texts, val_texts, train_labels, val_labels = train_test_split(
        raw_train_texts,
        raw_train_labels,
        test_size=config.val_ratio,
        stratify=raw_train_labels,
        random_state=config.seed
    )
```

从原始的加载的数据集中随机抽样出数据组成实际加载的数据集，使用 `limit__samples` 方法：

```
train_texts, train_labels = limit_samples(train_texts, train_labels, config.train_limit, config.seed)
val_texts, val_labels = limit_samples(val_texts, val_labels, config.val_limit, config.seed)
test_texts, test_labels = limit_samples(raw_test_texts, raw_test_labels, config.test_limit, config.seed)
```

(5). 清洗文本

程序依次处理训练集、验证集和测试集文本，去除 HTML 标签、统一小写、过滤特殊字符，并输出预处理样例。

```
print("\n==== 预处理示例 =====")
print("原始文本: ", train_texts_raw[0][:250])
print("清洗后文本: ", train_texts[0][:250])
print("分词结果: ", simple_tokenize(train_texts[0])[:30])

stats = describe_dataset(train_texts)
print("\n==== 训练集文本长度统计 =====")
print(
    f"平均长度: {stats['mean_length']:.2f}, 中位数: {stats['median_length']:.2f}, "
    f"最长: {stats['max_length']:.0f}, 最短: {stats['min_length']:.0f}"
)
```

清洗文本的函数 `clean_text` 具体定义如下所示：

```
def clean_text(text: str) -> str:
    """对原始文本做基础清洗。"""
    text = re.sub(r"<[^>]+>", " ", text)
    text = text.lower()
    text = re.sub(r"^[a-z0-9\s'.!?,;:()-]", " ", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text
```

```
(transformer-text-cls) PS F:\课设\机器学习及优化\作业五-2> python .\transformer_experiment.py
训练日志文件: outputs\training_20260514_172727.log
当前设备: cuda
实际使用数据集: IMDB
训练集大小: 12000, 验证集大小: 2000, 测试集大小: 5000
清洗训练集: 100% | 12000/12000 [00:01<00:00, 6161.77it/s]
清洗验证集: 100% | 2000/2000 [00:00<00:00, 6313.96it/s]
清洗测试集: 100% | 5000/5000 [00:00<00:00, 6025.97it/s]
```

(6). 构建词表

程序只使用训练集文本构建词表，设置最大词表大小为 20000，并保存到 outputs/vocab.json

```
# 构建词表
vocab = build_vocab(train_texts, config.max_vocab_size, config.min_freq)
with open(os.path.join(config.save_dir, "vocab.json"), "w", encoding="utf-8") as f:
    json.dump(vocab, f, ensure_ascii=False, indent=2)
print(f"词表大小: {len(vocab)}")
```

构建词表的函数定义如下：

```
def build_vocab(texts: List[str], max_vocab_size: int, min_freq: int) -> Dict[str, int]:
    """根据训练集文本构建词表。"""
    counter = Counter()
    for text in texts:
        counter.update(simple_tokenize(text))

    vocab_tokens = ["[PAD]", "[UNK]"]
    frequent_tokens = [token for token, freq in counter.most_common() if freq >= min_freq]
    vocab_tokens.extend(frequent_tokens[: max_vocab_size - len(vocab_tokens)])

    vocab = {token: idx for idx, token in enumerate(vocab_tokens)}
    return vocab
```

构建结果示例如下：

```
{
  "[PAD]": 0,
  "[UNK]": 1,
  "the": 2,
  ".": 3,
  ",": 4,
  "and": 5,
  "a": 6,
  "of": 7,
  "to": 8,
  "is": 9,
  "in": 10,
  "it": 11,
  "i": 12,
  "this": 13,
  "that": 14,
  "-": 15,
  "was": 16,
  "as": 17,
  "for": 18,
  "with": 19,
  "movie": 20,
  "but": 21,
```

==== 预处理示例 ====

```
原始文本: "Algie, the Miner" is one bad and unfunny silent comedy. The timing of the slapstick is completely off. This
is the kind of humor with certain sequences that make you wonder if they're supposed to be funny or not. However, the ac
tual quality of the f
清洗后文本: algie, the miner is one bad and unfunny silent comedy. the timing of the slapstick is completely off. this
is the kind of humor with certain sequences that make you wonder if they're supposed to be funny or not. however, the ac
tual quality of the fil
分词结果: ['algie', ',', 'the', 'miner', 'is', 'one', 'bad', 'and', 'unfunny', 'silent', 'comedy', '.', 'the', 'timing',
', 'of', 'the', 'slapstick', 'is', 'completely', 'off', '.', 'this', 'is', 'the', 'kind', 'of', 'humor', 'with', 'certain',
', 'sequences']
```

(7). 构建数据集和 DataLoader

input_ids 和 attention_mask, 再生成训练、验证、测试三个 DataLoader。

```
scratch_train_ds = ScratchTextDataset(train_texts, train_labels, vocab, config.max_length)
scratch_val_ds = ScratchTextDataset(val_texts, val_labels, vocab, config.max_length)
scratch_test_ds = ScratchTextDataset(test_texts, test_labels, vocab, config.max_length)

scratch_train_loader = DataLoader(scratch_train_ds, batch_size=config.batch_size, shuffle=True, **loader_kwargs)
scratch_val_loader = DataLoader(scratch_val_ds, batch_size=config.batch_size, shuffle=False, **loader_kwargs)
scratch_test_loader = DataLoader(scratch_test_ds, batch_size=config.batch_size, shuffle=False, **loader_kwargs)
```

==== 训练集文本长度统计 ====

平均长度: 265.20, 中位数: 197.00, 最长: 2732, 最短: 11
词表大小: 20000

(8). 初始化并训练 Scratch Transformer

程序创建 TransformerClassifier, 依次执行训练、验证、早停判断和最优模型保存, 最优权重保存到 outputs/models/scratch_transformer_best.pt。

```

class TransformerClassifier(nn.Module):
    """从零实现的 Transformer 编码器文本分类模型。"""

    def __init__(
        self,
        vocab_size: int,
        d_model: int,
        n_heads: int,
        n_layers: int,
        ff_dim: int,
        max_length: int,
        dropout: float,
        num_classes: int,
        pad_idx: int
    ):
        super().__init__()
        self.d_model = d_model
        self.embedding = nn.Embedding(vocab_size, d_model, padding_idx=pad_idx)
        self.positional_encoding = PositionalEncoding(d_model, max_length)
        self.layers = nn.ModuleList(
            [TransformerEncoderBlock(d_model, n_heads, ff_dim, dropout) for _ in range(n_layers)]
        )
        self.dropout = nn.Dropout(dropout)
        self.classifier = nn.Linear(d_model, num_classes)

    def forward(self, input_ids: torch.Tensor, attention_mask: torch.Tensor) -> torch.Tensor:
        x = self.embedding(input_ids) * math.sqrt(self.d_model)
        x = self.positional_encoding(x)
        x = self.dropout(x)

        for layer in self.layers:
            x = layer(x, attention_mask)

        # 使用 attention_mask 做掩码平均池化, 避免 PAD 对分类结果造成干扰
        mask = attention_mask.unsqueeze(-1).float()
        pooled = (x * mask).sum(dim=1) / mask.sum(dim=1).clamp(min=1e-9)
        logits = self.classifier(pooled)
        return logits

```

```

class PositionalEncoding(nn.Module):
    """手工实现正弦位置编码。"""

    def __init__(self, d_model: int, max_length: int):
        super().__init__()
        pe = torch.zeros(max_length, d_model)
        position = torch.arange(0, max_length, dtype=torch.float32).unsqueeze(1)
        div_term = torch.exp(torch.arange(0, d_model, 2, dtype=torch.float32) * (-math.log(10000.0) / d_model))
        pe[:, 0::2] = torch.sin(position * div_term)
        pe[:, 1::2] = torch.cos(position * div_term)
        self.register_buffer("pe", pe.unsqueeze(0))

```

可以看到前向传播的 forward 函数中体现了本实验中 transformer 模型的大致 pipeline，以下是关于 Encoder block 的完整定义：

```

class TransformerEncoderBlock(nn.Module):
    """单层 Transformer 编码器, 包含注意力、残差连接、层归一化和前馈网络。"""

    def __init__(self, d_model: int, n_heads: int, ff_dim: int, dropout: float):
        super().__init__()
        self.attn = MultiHeadSelfAttention(d_model, n_heads, dropout)
        self.ffn = FeedForwardNetwork(d_model, ff_dim, dropout)
        self.norm1 = nn.LayerNorm(d_model)
        self.norm2 = nn.LayerNorm(d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x: torch.Tensor, attention_mask: Optional[torch.Tensor] = None) -> torch.Tensor:
        attn_output = self.attn(x, attention_mask)
        x = self.norm1(x + self.dropout(attn_output))

        ffn_output = self.ffn(x)
        x = self.norm2(x + self.dropout(ffn_output))
        return x

```

Block 中的多个组件（多头自注意力、前馈神经网络）定义全由手工实现，如下所示：

```
class MultiHeadSelfAttention(nn.Module):
    """手工实现多头自注意力机制。"""

    def __init__(self, d_model: int, n_heads: int, dropout: float):
        super().__init__()
        assert d_model % n_heads == 0, "d_model 必须能被 n_heads 整除。"
        self.d_model = d_model
        self.n_heads = n_heads
        self.head_dim = d_model // n_heads

        self.q_proj = nn.Linear(d_model, d_model)
        self.k_proj = nn.Linear(d_model, d_model)
        self.v_proj = nn.Linear(d_model, d_model)
        self.out_proj = nn.Linear(d_model, d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x: torch.Tensor, attention_mask: Optional[torch.Tensor] = None) -> torch.Tensor:
        batch_size, seq_len, _ = x.size()

        q = self.q_proj(x).view(batch_size, seq_len, self.n_heads, self.head_dim).transpose(1, 2)
        k = self.k_proj(x).view(batch_size, seq_len, self.n_heads, self.head_dim).transpose(1, 2)
        v = self.v_proj(x).view(batch_size, seq_len, self.n_heads, self.head_dim).transpose(1, 2)

        scores = torch.matmul(q, k.transpose(-2, -1)) / math.sqrt(self.head_dim)

        if attention_mask is not None:
            mask = attention_mask.unsqueeze(1).unsqueeze(2)
            scores = scores.masked_fill(mask == 0, -1e9)

        attn_weights = torch.softmax(scores, dim=-1)
        attn_weights = self.dropout(attn_weights)

        context = torch.matmul(attn_weights, v)
        context = context.transpose(1, 2).contiguous().view(batch_size, seq_len, self.d_model)
        output = self.out_proj(context)
        return output
```

```
class FeedForwardNetwork(nn.Module):
    """前馈神经网络模块。"""

    def __init__(self, d_model: int, ff_dim: int, dropout: float):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(d_model, ff_dim),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(ff_dim, d_model)
        )

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)
```

以下是实现对 Transformer 模型训练的训练函数 fit_scratch_transformer:

```

def fit_scratch_transformer(
    model: nn.Module,
    train_loader: DataLoader,
    val_loader: DataLoader,
    config: Config,
    device: torch.device,
    save_path: str
) -> Dict[str, List[float]]:
    """训练从零实现的 Transformer 并执行验证和早停。"""
    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.AdamW(
        model.parameters(),
        lr=config.transformer_lr,
        weight_decay=config.transformer_weight_decay
    )
    early_stopping = EarlyStopping(config.early_stop_patience, save_path, mode="max")

    history = {"train_loss": [], "val_loss": [], "train_acc": [], "val_acc": []}

    for epoch in range(1, config.transformer_epochs + 1):
        train_loss, train_acc, train_f1 = train_one_epoch_scratch(model, train_loader, optimizer, criterion, device)
        val_loss, val_acc, val_f1, _, _ = evaluate_scratch(model, val_loader, criterion, device)

        history["train_loss"].append(train_loss)
        history["val_loss"].append(val_loss)
        history["train_acc"].append(train_acc)
        history["val_acc"].append(val_acc)

        print(
            f"[Scratch][Epoch {epoch:02d}] "
            f"train_loss={train_loss:.4f} train_acc={train_acc:.4f} train_f1={train_f1:.4f} | "
            f"val_loss={val_loss:.4f} val_acc={val_acc:.4f} val_f1={val_f1:.4f}"
        )

        should_stop, improved = early_stopping.step(val_f1, model)
        if improved:
            print(" 验证集 F1 提升，已保存当前最优模型。")
        if should_stop:
            print(" 触发早停，结束 Scratch Transformer 训练。")
            break

    model.load_state_dict(torch.load(save_path, map_location=device))
    return history

```

训练过程如下所示：

```

[Scratch][Epoch 01] train_loss=0.5408 train_acc=0.7220 train_f1=0.7259 | val_loss=0.4976 val_acc=0.7630 val_f1=0.7322
 验证集 F1 提升，已保存当前最优模型。
[Scratch][Epoch 02] train_loss=0.4369 train_acc=0.7975 train_f1=0.7989 | val_loss=0.4372 val_acc=0.8070 val_f1=0.8041
 验证集 F1 提升，已保存当前最优模型。
[Scratch][Epoch 03] train_loss=0.4031 train_acc=0.8182 train_f1=0.8193 | val_loss=0.4257 val_acc=0.8120 val_f1=0.8159
 验证集 F1 提升，已保存当前最优模型。
[Scratch][Epoch 04] train_loss=0.3795 train_acc=0.8289 train_f1=0.8301 | val_loss=0.4279 val_acc=0.8140 val_f1=0.8009
[Scratch][Epoch 05] train_loss=0.3521 train_acc=0.8459 train_f1=0.8463 | val_loss=0.4366 val_acc=0.8100 val_f1=0.8025
[Scratch][Epoch 06] train_loss=0.3431 train_acc=0.8534 train_f1=0.8537 | val_loss=0.4104 val_acc=0.8250 val_f1=0.8262
 验证集 F1 提升，已保存当前最优模型。
[Scratch][Epoch 07] train_loss=0.3235 train_acc=0.8612 train_f1=0.8621 | val_loss=0.4196 val_acc=0.8225 val_f1=0.8142
[Scratch][Epoch 08] train_loss=0.3075 train_acc=0.8702 train_f1=0.8703 | val_loss=0.4078 val_acc=0.8280 val_f1=0.8294
 验证集 F1 提升，已保存当前最优模型。
[Scratch][Epoch 09] train_loss=0.2955 train_acc=0.8776 train_f1=0.8780 | val_loss=0.4171 val_acc=0.8240 val_f1=0.8233
[Scratch][Epoch 10] train_loss=0.2839 train_acc=0.8811 train_f1=0.8811 | val_loss=0.4055 val_acc=0.8190 val_f1=0.8225

```

(9). 评估 Scratch Transformer

程序加载最优模型，在测试集上计算 Test Loss、Accuracy、F1 和混淆矩阵，并保存训练曲线和混淆矩阵图片。

```

scratch_test_loss, scratch_test_acc, scratch_test_f1, scratch_cm, _, _ = evaluate_scratch(
    scratch_model,
    scratch_test_loader,
    scratch_criterion,
    device
)

```

```

@torch.no_grad()
def evaluate_scratch(
    model: nn.Module,
    loader: DataLoader,
    criterion: nn.Module,
    device: torch.device
) -> Tuple[float, float, float, np.ndarray, List[int], List[int]]:
    """评估从零实现的 Transformer。"""
    model.eval()
    total_loss = 0.0
    y_true, y_pred = [], []

    for batch in tqdm(loader, desc="Scratch Eval", leave=False):
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        logits = model(input_ids, attention_mask)
        loss = criterion(logits, labels)

        total_loss += loss.item() * labels.size(0)
        preds = torch.argmax(logits, dim=1)

        y_true.extend(labels.cpu().tolist())
        y_pred.extend(preds.cpu().tolist())

    metrics = compute_metrics(y_true, y_pred)
    avg_loss = total_loss / len(loader.dataset)
    return avg_loss, metrics["accuracy"], metrics["f1"], metrics["confusion_matrix"], y_true, y_pred

```

```

plot_training_curves(
    scratch_history,
    os.path.join(figure_dir, "scratch_transformer_training_curves.png"),
    "Scratch Transformer"
)
plot_confusion_matrix(
    scratch_cm,
    ["Negative", "Positive"],
    os.path.join(figure_dir, "scratch_transformer_confusion_matrix.png"),
    "Scratch Transformer Confusion Matrix"
)

print("\n==== Scratch Transformer 测试集结果 =====")
print(f"test_loss={scratch_test_loss:.4f}, test_acc={scratch_test_acc:.4f}, test_f1={scratch_test_f1:.4f}")
print("confusion_matrix:")
print(scratch_cm)

```

评测结果如下所示：

```

==== Scratch Transformer 测试集结果 ====
test_loss=0.4165, test_acc=0.8200, test_f1=0.8199
confusion_matrix:
[[2051  449]
 [ 451 2049]]

```

(10). 加载并微调 BERT

程序使用 `AutoTokenizer` 和 `AutoModelForSequenceClassification` 加载 `bert-base-uncased`，构建 BERT 数据集和 `DataLoader`，然后训练 3 个 epoch


```

bert_tokenizer = AutoTokenizer.from_pretrained(config.bert_model_name, use_fast=True)
bert_model = AutoModelForSequenceClassification.from_pretrained(config.bert_model_name, num_labels=2).to(device)

bert_train_ds = BertTextDataset(train_texts, train_labels, bert_tokenizer, config.max_length)
bert_val_ds = BertTextDataset(val_texts, val_labels, bert_tokenizer, config.max_length)
bert_test_ds = BertTextDataset(test_texts, test_labels, bert_tokenizer, config.max_length)

bert_train_loader = DataLoader(bert_train_ds, batch_size=config.bert_batch_size, shuffle=True, **loader_kwargs)
bert_val_loader = DataLoader(bert_val_ds, batch_size=config.bert_batch_size, shuffle=False, **loader_kwargs)
bert_test_loader = DataLoader(bert_test_ds, batch_size=config.bert_batch_size, shuffle=False, **loader_kwargs)

bert_model_path = os.path.join(model_dir, "bert_best.pt")
bert_history = fit_bert(
    bert_model,
    bert_train_loader,
    bert_val_loader,
    config,
    device,
    bert_model_path
)

```

微调过程如下所示：

```

[BERT][Epoch 01] train_loss=0.3433 train_acc=0.8448 train_f1=0.8426 | val_loss=0.2438 val_acc=0.8995 val_f1=0.9030
验证集 F1 提升，已保存当前最优 BERT 模型。
[BERT][Epoch 02] train_loss=0.1668 train_acc=0.9464 train_f1=0.9465 | val_loss=0.2835 val_acc=0.9165 val_f1=0.9170
验证集 F1 提升，已保存当前最优 BERT 模型。
[BERT][Epoch 03] train_loss=0.0809 train_acc=0.9784 train_f1=0.9784 | val_loss=0.3850 val_acc=0.9140 val_f1=0.9143

```

(11). 评估 BERT

程序在测试集上计算 BERT 的 Test Loss、Accuracy、F1 和混淆矩阵，并保存对应图像

```

bert_test_loss, bert_test_acc, bert_test_f1, bert_cm, _, _ = evaluate_bert(
    bert_model,
    bert_test_loader,
    device
)

plot_training_curves(
    bert_history,
    os.path.join(figure_dir, "bert_training_curves.png"),
    "BERT Fine-tuning"
)

plot_confusion_matrix(
    bert_cm,
    ["Negative", "Positive"],
    os.path.join(figure_dir, "bert_confusion_matrix.png"),
    "BERT Confusion Matrix"
)

print("\n==== BERT 测试集结果 =====")
print(f"test_loss={bert_test_loss:.4f}, test_acc={bert_test_acc:.4f}, test_f1={bert_test_f1:.4f}")
print("confusion_matrix:")
print(bert_cm)

```

评测结果如下所示：

```

==== BERT 测试集结果 =====
test_loss=0.2578, test_acc=0.9184, test_f1=0.9190
confusion_matrix:
[[2276  224]
 [ 184 2316]]

```

(12). 训练并评估传统基线模型

使用 TfidfVectorizer(max_features=10000) 将文本转为 TF-IDF 特征，再使用 LogisticRegression 训练分类器，最后在测试集上计算 Accuracy、F1 和混淆矩阵，并保存混淆矩阵图像

```

baseline_metrics = train_tfidf_logistic_regression(train_texts, train_labels, test_texts, test_labels)
plot_confusion_matrix(
    baseline_metrics["confusion_matrix"],
    ["Negative", "Positive"],
    os.path.join(figure_dir, "tfidf_lr_confusion_matrix.png"),
    "TF-IDF + Logistic Regression Confusion Matrix"
)

print("\n==== TF-IDF + LogisticRegression 测试集结果 =====")
print(f"test_acc={baseline_metrics['accuracy']:.4f}, test_f1={baseline_metrics['f1']:.4f}")
print("confusion_matrix:")
print(baseline_metrics["confusion_matrix"])

```

评测结果如下所示：

```

===== TF-IDF + LogisticRegression 测试集结果 =====
test_acc=0.8656, test_f1=0.8662
confusion_matrix:
[[2152  348]
 [ 324 2176]]

```

五、实验结果

(1). 模型评估方法的综合对比

```

Hold-out evaluation:
R2  = 0.6074
MSE = 0.5197

5-fold CV results:
R2  mean = 0.6038, std = 0.0113
MSE mean = 0.5274, std = 0.0099

10-fold CV results:
R2  mean = 0.6035, std = 0.0184
MSE mean = 0.5275, std = 0.0216

Bootstrap (OOB) results:
R2  mean = 0.4400, std = 0.9235
MSE mean = 0.7442, std = 1.2190

Learning-curve conclusion: High bias (underfitting)

```

评估方法	具体设置	R2 均值	R2 标准差	MSE 均值	MSE 标准差	说明
留出法 (Hold-out)	70% 训练集 / 30% 测试集	0.6074	-	0.5197	-	单次划分，只能得到一次测试结果
5 折交叉验证	K = 5	0.6038	0.0113	0.5274	0.0099	多次划分后取平均，结果较稳定
10 折交叉验证	K = 10	0.6035	0.0184	0.5275	0.0216	均值与 5 折几乎一致，但本次波动更大

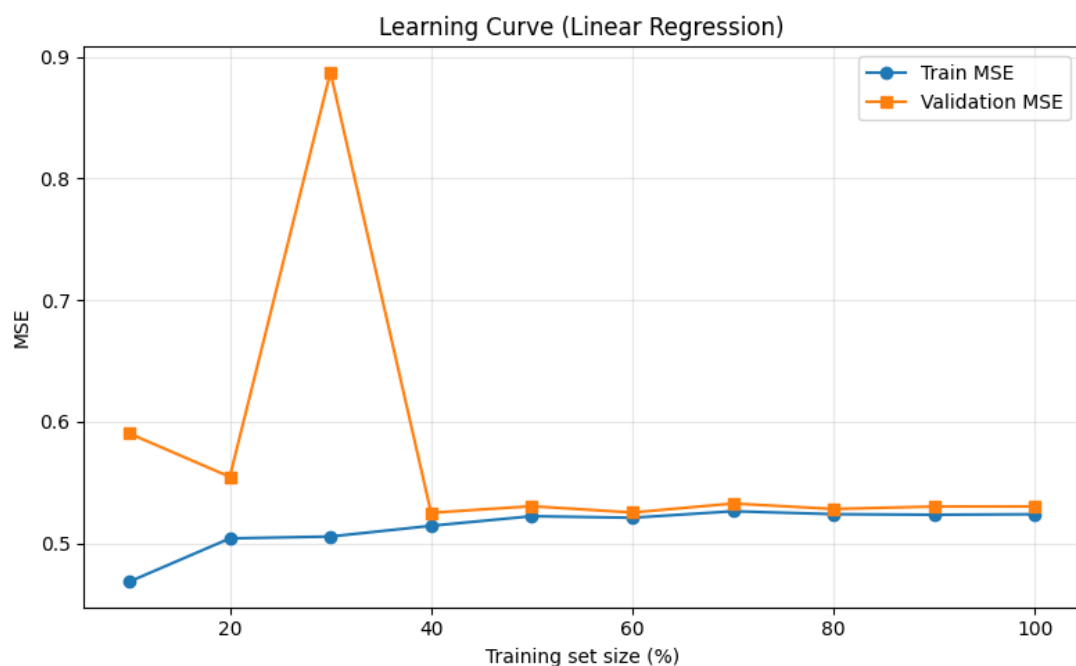
Bootstrap 自助法	B = 100, OOB 评 估	0.4400	0.9235	0.7442	1.2190	波动明显, 结果 稳定性较差
------------------	------------------------	--------	--------	--------	--------	-------------------

从整体结果看，留出法、5 折交叉验证和 10 折交叉验证得到的模型性能比较接近。留出法的 $R^2 = 0.6074$ ，略高于 5 折交叉验证的 0.6038 和 10 折交叉验证的 0.6035；留出法的 MSE 为 0.5197，也略低于交叉验证中的约 0.527。这说明本次 70%/30% 的随机划分结果相对有利，但差距很小，模型整体表现基本一致。

5 折和 10 折交叉验证的均值几乎相同，说明线性回归模型在该数据集上的平均泛化能力比较稳定， R^2 大约维持在 0.60， MSE 大约维持在 0.527。不过本次实验中，10 折交叉验证的标准差反而更大： R^2 标准差从 5 折的 0.0113 增加到 0.0184， MSE 标准差从 0.0099 增加到 0.0216。这可能是因为 10 折时每个验证集样本更少，单折结果更容易受局部样本分布影响。

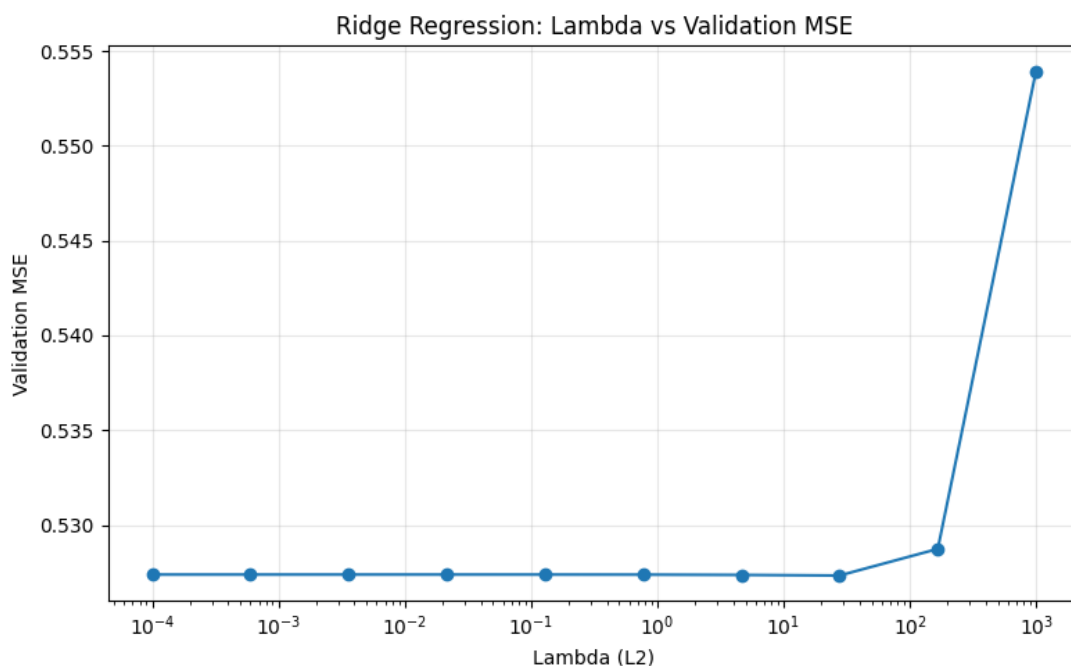
Bootstrap 自助法的结果与前两类方法差异较大。其 R^2 均值只有 0.4400，明显低于留出法和交叉验证； MSE 均值为 0.7442，明显高于其他方法。同时，Bootstrap 的标准差非常大，说明不同次有放回采样得到的 OOB 评估结果波动剧烈。因此，在本次实验中，Bootstrap 更适合作为模型评估不确定性的参考，而不适合作为最主要的性能结论。

综合来看，本实验中最可靠的主要结论应以 K 折交叉验证为主，留出法作为快速对照，Bootstrap 作为稳定性和不确定性分析的补充。



从图中可以看出，随着训练集比例从 10% 增加到 100%，训练 MSE 略有上升，大约从 0.47 上升到 0.52 左右；验证 MSE 则从较高位置逐渐下降，大约从 0.59 降到 0.53 左右。训练误差和验证误差在训练集比例达到 50% 以后逐渐接近，最终二者差距较小。

这说明模型并没有明显过拟合。如果是过拟合，通常会表现为训练误差很低、验证误差明显较高；但本图中两条曲线最终比较接近，且误差整体仍偏高，因此更符合“高偏差 / 欠拟合”的特征。也就是说，当前线性回归模型表达能力有限，单纯增加训练数据带来的提升可能有限，更有效的改进方向是加入非线性特征、交互特征，或尝试更复杂的模型。



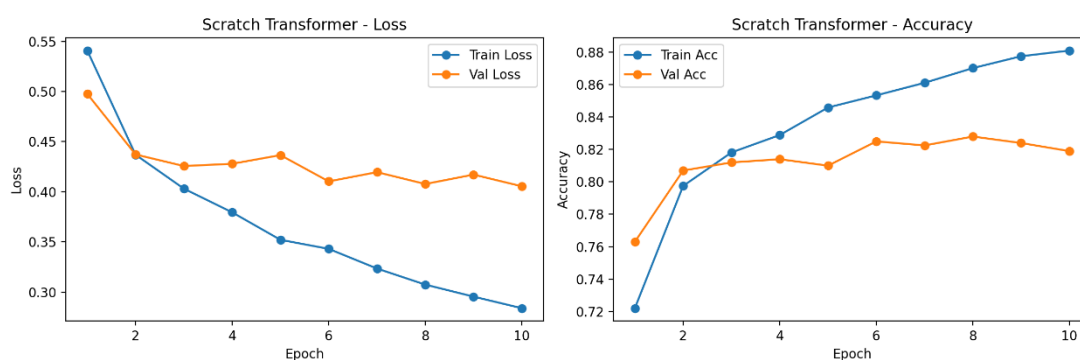
从图中可以看出，当 λ 位于较小范围时，例如 10^{-4} 到 10^1 ，验证 MSE 基本保持在约 0.527 附近，变化不明显。这说明较弱的 L2 正则化并没有显著改善模型性能。

当 λ 增大到 10^2 及以上时，验证 MSE 明显上升，说明过强的正则化限制了模型参数大小，使模型更加简单，进一步加重了欠拟合问题。

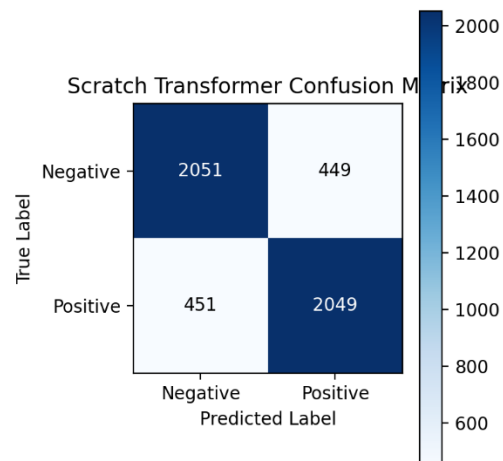
结合学习曲线的结论可以看出，当前模型主要问题不是过拟合，而是欠拟合。因此，正则化对性能提升有限；过强正则化反而会降低模型表达能力，使验证误差变大。

(2). Transformer 编码器文本分类任务

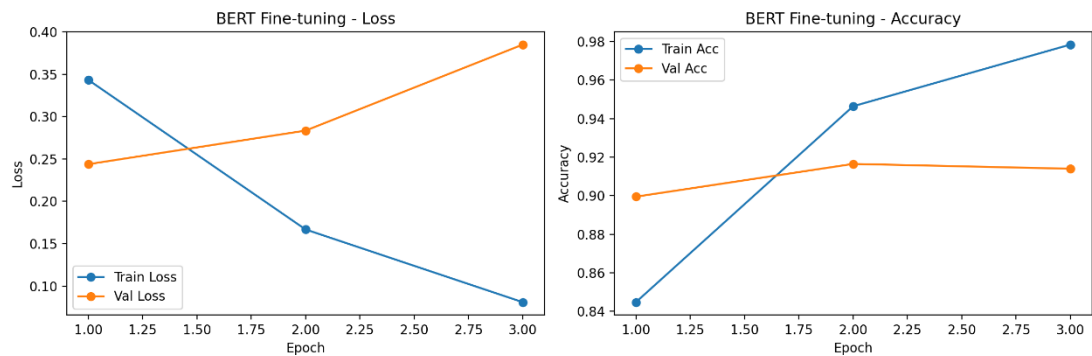
从零实现 Transformer 的训练曲线如下图所示，左图为训练集和验证集 loss，右图为训练集和验证集 accuracy。



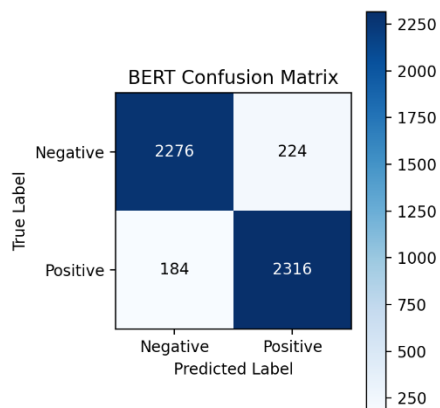
模型在测试集上的混淆矩阵如下



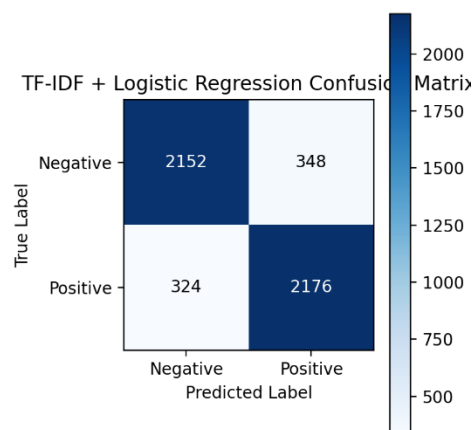
BERT 微调模型的训练曲线如下：



BERT 在测试集上的混淆矩阵如下：



TF-IDF + LogisticRegression 模型在测试集上的混淆矩阵如下：



汇总的三个模型在 IMDB 上的表现如下：

模型	Accuracy	F1
Scratch Transformer	0.8200	0.8199
BERT Fine-tuning	0.9196	0.9201
TF-IDF+LogisticRegression	0.8656	0.8662

从对比结果可以看出，BERT 微调模型取得了最佳性能；传统 TF-IDF + LogisticRegression 基线优于从零实现的轻量 Transformer；从零实现 Transformer 虽然能学习上下文信息，但受训练数据规模、模型初始化和训练轮数限制，性能低于另外两种方法。

- BERT 的优势主要来自大规模预训练。`bert-base-uncased` 已经在大规模英文语料上学习到了丰富的词法、句法和语义知识，微调时只需要将这些通用语言表示迁移到 IMDB 情感分类任务上。相比之下，从零实现的 Transformer 只使用本次实验中的 12000 条训练样本进行学习，难以在有限数据上获得与预训练模型同等质量的语言表示。BERT 还使用 WordPiece 子词分词，可以更好地处理低频词、派生词和未登录词。从零实现模型采用简单正则分词和当前训练集词表，遇到低频词或词表外词时只能映射为 [UNK]，语义信息损失更明显。此外，BERT 的模型规模更大，结构更深，预训练初始化也更稳定。本实验中的从零实现 Transformer 设置为 `d_model=128`、`n_layers=4`，表达能力与 BERT 相比有限。
- 从零实现 Transformer 的优势是可以通过自注意力建模 token 之间的上下文关系，理论上能捕捉否定、转折和长距离依赖。例如在 “not good” 或 “good, but boring” 这类表达中，上下文关系对情感判断很重要。不过本次实验中，TF-IDF + LogisticRegression 的 Accuracy 为 0.8656，高于 Scratch Transformer 的 0.8200。原因是 IMDB 情感分类中存在大量强情感词，TF-IDF 能稳定捕捉这些词频和逆文档频率特征；逻辑回归参数少、训练稳定，对中小规模数据非常友好。从零训练的 Transformer 参数更多，对数据量和训练策略更敏感。如果没有预训练、数据增强或更长训练，模型容易无法充分学习有效的语言模式。因此在本实验设置下，传统基线虽然结构简单，却取得了更好的泛化表现。

六、实验体会

(1). 模型评估方法的综合对比

通过本次实验，我更加清楚地理解了不同模型评估方法的特点。留出法实现简单、结果直观，但只依赖一次数据划分，存在一定偶然性；K 折交叉验证通过多次划分并取平均，结果更加稳定，本实验中 5 折和 10 折交叉验证的结果都与留出法接近，说明模型的整体泛化能力较为一致。Bootstrap 自助法虽然可以利用 OOB 样本进行评估，但本次结果波动较大，说明在分析模型性能时不能只看均值，也要关注标准差和稳定性。

学习曲线和正则化实验让我进一步认识到，当前线性回归模型主要问题是欠拟合而不是过拟合。训练误差和验证误差逐渐接近，但整体误差仍然偏高，说明模型表达能力有限；同时，较强的 L2 正则化会使验证误差上升，进一步说明单纯正则化不能有效提升该模型。总体来看，本实验让我认识到模型评估需要结合多种方法、多个指标和图像结果综合判断，才能更准确地分析模型问题并提出改进方向。

(2). 从零实现 Transformer 编码器分类模型

本实验完整实现并比较了三类文本分类方法：从零实现的 Transformer 编码器、BERT 微调模型和 TF-IDF + LogisticRegression 传统基线。实验结果表明，BERT 微调在 IMDB 情感分类任务上表现最佳，Accuracy 达到 0.9196，说明预训练语言模型在自然语言理解任务中具有明显优势。

通过手工实现 Transformer 编码器，可以更直观地理解自注意力、多头注意力、位置编码、前馈网络、残差连接和层归一化的作用。虽然从零实现模型的性能不如 BERT，也低于本次 TF-IDF 基线，但它清晰展示了现代 NLP 模型的基础结构，具有很好的学习价值。

综合来看，本实验加深了对 Transformer 编码器结构和文本分类流程的理解，也说明了在实际任务中应根据数据规模、计算资源和性能需求选择合适的模型方案。

教师评语：

指导教师：

日 期：